

Data Structures & Algorithms

BCA — Semester 3 — 2020 — Answer Sheet

Subject Code: CACS201

Group B

Attempt any SIX questions. [6×5=30]

2. What is data structure? Explain different operations to be performed on data structure.

[1+4]

Answer: Data Structure: A data structure is a specialized format for organizing, processing, retrieving and storing data. It defines the relationship between the data elements and the operations that can be performed on them. Data structures are essential for designing efficient algorithms and are classified as linear (arrays, linked lists, stacks, queues) and non-linear (trees, graphs).

Different Operations on Data Structures:

1. **Traversal:** Visiting each element of the data structure exactly once. For example, printing all elements of an array or linked list.
2. **Insertion:** Adding a new element to the data structure at a specific position. For example, inserting a node in a linked list or an element in an array.
3. **Deletion:** Removing an element from the data structure. For example, deleting a node from a tree or an element from a stack.
4. **Searching:** Finding the location of a specific element within the data structure. Linear search and binary search are common searching techniques.
5. **Sorting:** Arranging the elements in a specific order (ascending or descending). Examples include bubble sort, quick sort, merge sort, and heap sort.
6. **Merging:** Combining two sorted data structures into a single sorted data structure.
7. **Updating:** Modifying the value of an existing element at a given position.

3. Define greedy algorithm and heuristic algorithm. Briefly explain Big-Oh notation.

[3+2]

Answer: Greedy Algorithm: A greedy algorithm is an algorithmic paradigm that follows the problem-solving heuristic of making the locally optimal choice at each stage with the hope of finding a global optimum. It makes the choice that seems best at the moment without considering future consequences. Greedy algorithms are used when a problem has the properties of greedy choice property and optimal substructure. Examples include Dijkstra's shortest path algorithm, Kruskal's and Prim's minimum spanning tree algorithms, and the fractional knapsack problem.

Heuristic Algorithm: A heuristic algorithm is a technique designed to solve a problem more quickly when classic methods are too slow, or to find an approximate solution when classic methods fail to find any exact solution. Heuristics use practical methods, experience-based techniques, or rules of thumb rather than formal proofs. They are commonly used in artificial intelligence, optimization problems, and search algorithms where finding an exact solution is computationally infeasible. Examples include A* search algorithm, genetic algorithms, and simulated annealing.

Big-Oh Notation: Big-Oh notation (O) is a mathematical notation used in computer science to describe the performance or complexity of an algorithm. Specifically, it describes the upper bound of the growth rate of an algorithm's time or space requirement as the input size increases.

Common Big-Oh complexities:

Notation	Name	Example
$O(1)$	Constant time	Accessing an array element
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, Quick sort
$O(n^2)$	Quadratic	Bubble sort, Selection sort
$O(2^n)$	Exponential	Recursive Fibonacci

4. What is circular queue? Write an algorithm to insert an item in circular queue.

[2+3]

Answer: Circular Queue: A circular queue is a linear data structure that follows the FIFO (First In First Out) principle, where the last position is connected back to the first position to form a circle. It is also known as a ring buffer. In a circular queue, when the rear reaches the end of the array, it wraps around to the beginning if there are empty slots. This overcomes the limitation of a simple queue where space before the front pointer cannot be reused even if the queue is not full.

Algorithm to Insert an Item in Circular Queue:

Algorithm CQINSERT(Queue, FRONT, REAR, MAX, ITEM)
1. If (FRONT == 0 AND REAR == MAX - 1) OR (FRONT == REAR + 1) then Print "Queue Overflow" Exit
2. If FRONT == -1 then // Queue is empty FRONT = 0 REAR = 0
Else if REAR == MAX - 1 then // Wrap around REAR = 0
Else REAR = REAR + 1
3. Queue[REAR] = ITEM
4. Print "Item inserted successfully"
5. Exit

5. How does AVL tree differ from BST? Construct an AVL tree from following data: 35, 56, 68, 65, 44, 41, 31, 49, 20.

[1+4]

Answer: Difference between AVL Tree and BST:

Aspect	Binary Search Tree (BST)	AVL Tree
Balance	May become unbalanced (skewed)	Always balanced (self-balancing)
Height	Can be $O(n)$ in worst case	Always $O(\log n)$
Rotations	No rotations needed	Uses rotations (LL, RR, LR, RL) to maintain balance
Balance Factor	Not considered	Must be -1, 0, or +1 for every node
Search Time	$O(n)$ worst case	$O(\log n)$ guaranteed
Insertion/Deletion	Simple, no rebalancing	Complex, requires rebalancing

AVL Tree Construction with data: 35, 56, 68, 65, 44, 41, 31, 49, 20

Step 1: Insert 35 → Tree: 35

Step 2: Insert 56 → 35's right → Tree: 35(R:56) — Balanced (BF= -1)

Step 3: Insert 68 → 56's right → BF(35)= -2 → **RR Rotation** at 35

After RR: 56 is root, 35 left, 68 right

Step 4: Insert 65 → 68's left → Tree balanced (BF values: 56:0, 35:0, 68:-1, 65:0)

Step 5: Insert 44 → 35's right → Balanced

Step 6: Insert 41 → 44's left → BF(35)= +2, BF(44)= -1 → **LR Rotation** at 35

After LR: 41 becomes root of subtree, 35 left, 44 right

Step 7: Insert 31 → 35's left → Balanced

Step 8: Insert 49 → 44's right → Balanced

Step 9: Insert 20 → 31's left → BF(41)= +2, BF(35)= +2 → **LL Rotation** at 41

After LL: 35 becomes root of subtree, 31 left, 41 right, 20 is left of 31

Final AVL Tree Structure:

56 / 41 68 / \ / 35 44 65 / \ 31 49 (none) / 20

6. What is B-tree? Create a B-tree of order 4 using following data: 6, 4, 22, 10, 2, 14, 3, 8, 11, 13, 5, 9.

[2+3]

Answer: B-Tree: A B-tree is a self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. It is a generalization of a binary search tree in which a node can have more than two children. A B-tree of order m (or degree m) has the following properties:

- All leaf nodes are at the same level.
- Every node except root must have at least $\lceil m/2 \rceil - 1$ keys and at most $m - 1$ keys.
- Every internal node has at most m children.
- The root has at least 2 children if it is not a leaf node.
- Keys within a node are stored in ascending order.

B-Tree of Order 4 (max 3 keys per node, max 4 children):

Data: 6, 4, 22, 10, 2, 14, 3, 8, 11, 13, 5, 9

Step 1: Insert 6 → [6]

Step 2: Insert 4 → [4, 6]

Step 3: Insert 22 → [4, 6, 22] (full)

Step 4: Insert 10 → Node overflows. Split: median 6 goes up as root.

Root: [6], Left: [4], Right: [10, 22]

Step 5: Insert 2 → [2, 4] (left child)

Step 6: Insert 14 → [10, 14, 22] (right child — full)

Step 7: Insert 3 → [2, 3, 4] (left child — full)

Step 8: Insert 8 → Goes to left of right child? No, $8 < 10$, so new child between 4 and 6? Let's trace properly with splits.

After insert 3, left child [2,3,4] is full. Insert 8: $8 > 6$, goes to right subtree. $8 < 10$, so new left child of [10,14,22]? No, $8 < 10$, so goes to position before 10. Node becomes [8,10,14,22] — overflow! Split: median 14 promoted to root.

Root: [6, 14], Children: [2,3,4], [8,10], [22]

Step 9: Insert 11 → $11 > 6$, $11 < 14$, goes to middle child [8,10] → [8,10,11]

Step 10: Insert 13 → [8,10,11,13] — overflow! Split: median 10 or 11. Let's use 10 or 11. Split at 10: [8] and [11,13], promote 10 to root. Root becomes [6,10,14] — but max 3 keys, so root overflows!

Split root: median 10 promoted to new root. New root: [10], Left: [6], Right: [14].

Now reorganize children: [2,3,4] under [6], [8] under [6], [11,13] under [14], [22] under [14].

Step 11: Insert 5 → $5 < 10$, $5 < 6$, goes to [2,3,4] → [2,3,4,5] — overflow! Split: promote 3 or 4. Split at 3: [2] and [4,5], promote 3 to [6] → [3,6] (max 3 OK).

Children of [3,6]: [2], [4,5], [8]

Step 12: Insert 9 → $9 < 10$, $9 > 6$, goes to [8] → [8,9]

Final B-Tree of Order 4:

[10] / [3, 6] [14] / | \ / [2] [4,5] [8,9] [11,13] [22]

7. What is binary search? Write an algorithm to search an item using binary search.

[2+3]

Answer: Binary Search: Binary search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing in half the portion of the list that could contain the item, until you've narrowed down the possible locations to just one. The time complexity of binary search is $O(\log n)$, making it much faster than linear search ($O(n)$) for large datasets. Binary search requires the array to be sorted beforehand.

Algorithm for Binary Search:

Algorithm `BINARY_SEARCH(A, N, ITEM)` // A is a sorted array of N elements // ITEM is the element to be searched
1. Set $LB = 0$, $UB = N - 1$
2. While $LB \leq UB$ do $MID = (LB + UB) / 2$
If $A[MID] == ITEM$ then Print "Item found at index", MID Exit
Else if $A[MID] < ITEM$ then $LB = MID + 1$
Else $UB = MID - 1$
3. Print "Item not found"
4. Exit

Example: Search for 23 in sorted array [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

- Iteration 1: $LB=0$, $UB=9$, $MID=4$, $A[4]=16 < 23 \rightarrow LB=5$
- Iteration 2: $LB=5$, $UB=9$, $MID=7$, $A[7]=56 > 23 \rightarrow UB=6$
- Iteration 3: $LB=5$, $UB=6$, $MID=5$, $A[5]=23 == 23 \rightarrow$ **Found at index 5!**

8. What is graph? Explain Kruskal's algorithm to construct minimum spanning tree with example.

[1+4]

Answer: Graph: A graph is a non-linear data structure consisting of a finite set of vertices (also called nodes) and a set of edges that connect pairs of vertices. Graphs are used to represent networks, relationships, and paths. A graph G is formally defined as $G = (V, E)$, where V is the set of vertices and E is the set of edges. Graphs can be directed (edges have direction) or undirected, weighted (edges have costs) or unweighted.

Kruskal's Algorithm for Minimum Spanning Tree (MST):

Kruskal's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected, undirected, weighted graph. The MST is a subset of edges that connects all vertices with the minimum total edge weight, without any cycles.

Algorithm:

1. Sort all edges in non-decreasing order of their weight.
2. Initialize an empty set for the MST.
3. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far using Union-Find.
4. If cycle is not formed, include this edge. Otherwise, discard it.
5. Repeat step 3 until there are $(V-1)$ edges in the MST.

Example: Consider a graph with vertices A, B, C, D, E and edges:

Edge A-B B-C A-C C-D B-D C-E Weight 2 3 4 5 6 7

Step 1: Sort edges by weight: D-E(1), A-B(2), B-C(3), A-C(4), C-D(5), B-D(6), C-E(7)

Step 2: Pick D-E(1) — no cycle $\rightarrow$ $MST = \{D-E\}$

Step 3: Pick A-B(2) — no cycle $\rightarrow$ $MST = \{D-E, A-B\}$

Step 4: Pick B-C(3) — no cycle $\rightarrow$ $MST = \{D-E, A-B, B-C\}$

Step 5: Pick A-C(4) — forms cycle (A-B-C-A) $\rightarrow$ **Reject**

Step 6: Pick C-D(5) — no cycle $\rightarrow$ $MST = \{D-E, A-B, B-C, C-D\}$

Now we have 4 edges ($V-1 = 5-1 = 4$). Stop.

Total weight of MST = 1 + 2 + 3 + 5 = 11

Group C

Attempt any TWO questions. [2×10=20]

9. Define stack. List the applications of stack. Trace the algorithm to convert infix to postfix with following infix expression $((A+B)-C*D/E)*(H-I)*F+G$ and evaluate the obtained postfix expression with following values: $A=4, B=2, C=4, D=3, E=8, F=2, G=3, H=5, I=1$

[1+1+4+4]

Answer: Stack: A stack is a linear data structure that follows the LIFO (Last In First Out) principle. Elements can only be inserted and deleted from one end called the top. The main operations are PUSH (insert) and POP (delete).

Applications of Stack:

1. Expression evaluation and conversion (infix to postfix/prefix)
2. Function call management (call stack in programming)
3. Undo/Redo operations in text editors
4. Backtracking algorithms
5. Parenthesis matching
6. Browser history navigation
7. Memory management (recursion)

Infix to Postfix Conversion:

Expression: $((A+B)-C*D/E)*(H-I)*F+G$

SymbolStackPostfix (((((A((A +((+A B((+AB)(AB+ -(-AB+ C(-AB+C *(-*AB+C D(-*AB+CD /(-/AB+CD* E(-/AB+CD*E)AB+CD*E/- **AB+CD*E/- (*(AB+CD*E/- H*(AB+CD*E/-H -*(-AB+CD*E/-H I*(-AB+CD*E/-HI)*AB+CD*E/-HI- **AB+CD*E/-HI-* F*AB+CD*E/-HI-*F ++AB+CD*E/-HI-*F* G+AB+CD*E/-HI-*F*G EndAB+CD*E/-HI-*F*G+

Postfix Expression: $AB+CD*E/-HI-*F*G+$

Evaluation ($A=4, B=2, C=4, D=3, E=8, F=2, G=3, H=5, I=1$):

SymbolOperationStack 4Push 44 2Push 24, 2 +4+2=66 4Push 46, 4 3Push 36, 4, 3 *4*3=126, 12 8Push 86, 12, 8 /12/8=1.56, 1.5 -6-1.5=4.54.5 5Push 54.5, 5 1Push 14.5, 5, 1 -5-1=44.5, 4 *4.5*4=1818 2Push 218, 2 *18*2=3636 3Push 336, 3 +36+3=3939

Final Result = 39

10. What is double linked list? How does it differ from circular linked list? Write an algorithm or function to add a node at the beginning and end of double linked list.

[1+1+4+4]

Answer: Doubly Linked List: A doubly linked list is a type of linked list in which each node contains three parts: data, a pointer to the next node, and a pointer to the previous node. This allows traversal in both forward and backward directions.

Difference between Doubly Linked List and Circular Linked List:

Aspect	Doubly Linked List	Circular Linked List
Structure	Each node has next and prev pointers	Last node points back to first node
Traversal	Bidirectional (forward and backward)	Unidirectional or bidirectional depending on type
End condition	Last node's next is NULL	Last node's next points to head
Memory	Requires extra memory for prev pointer	No extra pointer needed
Applications	Browser history, undo functionality	Round-robin scheduling, multiplayer games

Algorithm to Add Node at Beginning of Doubly Linked List:

Algorithm INSERT_BEGINNING(head, data) 1. Create a new node NEW_NODE 2. NEW_NODE.data = data 3. NEW_NODE.prev = NULL 4. NEW_NODE.next = head 5. If head != NULL then head.prev = NEW_NODE 6. head = NEW_NODE 7. Exit

Algorithm to Add Node at End of Doubly Linked List:

Algorithm INSERT_END(head, data) 1. Create a new node NEW_NODE 2. NEW_NODE.data = data 3. NEW_NODE.next = NULL 4. If head == NULL then NEW_NODE.prev = NULL head = NEW_NODE Exit 5. Set TEMP = head 6. While TEMP.next != NULL do TEMP = TEMP.next 7. TEMP.next = NEW_NODE 8. NEW_NODE.prev = TEMP 9. Exit

C Function Implementation:

```
struct Node { int data; struct Node* next; struct Node* prev; }; void insertBeginning(struct Node** head, int data) { struct Node* new_node = (struct Node*)malloc(sizeof(struct Node)); new_node->data = data; new_node->prev = NULL; new_node->next = *head; if (*head != NULL) (*head)->prev = new_node; *head = new_node; } void insertEnd(struct Node** head, int data) { struct Node* new_node = (struct Node*)malloc(sizeof(struct Node)); struct Node* temp = *head; new_node->data = data; new_node->next = NULL; if (*head == NULL) { new_node->prev = NULL; *head = new_node; return; } while (temp->next != NULL) temp = temp->next; temp->next = new_node; new_node->prev = temp; }
```

11. What is heap? Differentiate between min heap and max heap. Sort the following data in ascending order by heap sort method: 2, 9, 3, 12, 15, 8, 11.

[2+2+6]

Answer: Heap: A heap is a specialized tree-based data structure that satisfies the heap property. It is a complete binary tree where every level is fully filled except possibly the last level, which is filled from left to right. Heaps are commonly used to implement priority queues and in sorting algorithms like heap sort.

Difference between Min Heap and Max Heap:

Aspect	Min Heap	Max Heap
Property	Parent node is less than or equal to its children	Parent node is greater than or equal to its children
Root	Contains the minimum element	Contains the maximum element
Usage	Finding smallest element, Dijkstra's algorithm	Finding largest element, heap sort
Insertion	New element bubbled up if smaller than parent	New element bubbled up if larger than parent
Deletion	Remove root (min), replace with last, heapify down	Remove root (max), replace with last, heapify down

Heap Sort in Ascending Order (using Max Heap):

Data: 2, 9, 3, 12, 15, 8, 11

Step 1: Build Max Heap

Initial array: [2, 9, 3, 12, 15, 8, 11]

Heapify from last non-leaf node (index 2, value 3):

- Heapify 3: children 8, 11 → swap 3 with 11 → [2, 9, 11, 12, 15, 8, 3]
- Heapify 9: children 12, 15 → swap 9 with 15 → [2, 15, 11, 12, 9, 8, 3]
- Heapify 2: children 15, 11 → swap 2 with 15 → [15, 2, 11, 12, 9, 8, 3]

Now heapify 2: children 12, 9 → swap 2 with 12 → [15, 12, 11, 2, 9, 8, 3]

Then heapify 2: children none (leaf) → done.

Max Heap: [15, 12, 11, 2, 9, 8, 3]

Step 2: Heap Sort (extract elements one by one)

Step Swap root with last
Swap 15 and 3 → [3,12,11,2,9,8,15] Heapify [3,12,11,2,9,8] → [12,9,11,2,3,8][15]
Swap 12 and 8 → [8,9,11,2,3,12] Heapify [8,9,11,2,3] → [11,9,8,2,3][12,15]
Swap 11 and 3 → [3,9,8,2,11] Heapify [3,9,8,2] → [9,3,8,2][11,12,15]
Swap 9 and 2 → [2,3,8,9] Heapify [2,3,8] → [8,3,2][9,11,12,15]
Swap 8 and 2 → [2,3,8] Heapify [2,3] → [3,2][8,9,11,12,15]
Swap 3 and 2 → [2,3] Heapify [2] → [2][3,8,9,11,12,15]
Only 2 left [2,3,8,9,11,12,15]

Sorted Array (Ascending): [2, 3, 8, 9, 11, 12, 15]

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.